DBSR: An Efficient Storage Format for Vectorizing Sparse Triangular Solvers on Structured Grids

Xiaojian Yang, Shengguo Li, Fan Yuan, Dezun Dong

概要

背景

- さまざまなシミュレーションに用いられる SpTRSV（$L\mathbf{x} = \mathbf{b}$ を解く）
- 単純な並列化が難しい

目的

- SpTRSV の高速化
- グリッドの問題に対する最適化

手法

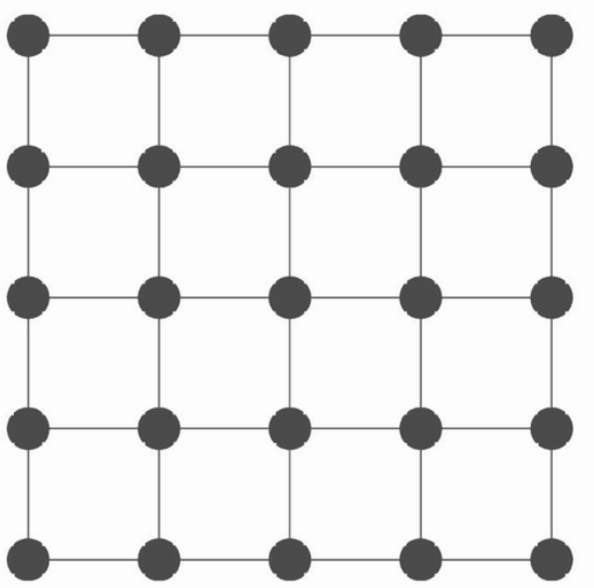
- SIMD の活用
- *Diagonal Block Compressed Sparse Row* (DBSR) 形式

結果

- MG法の実行時間が 1.19 倍から 1.24 倍高速化（他の最新の実装比）
- ILU(0) を平均 1.17 倍から 1.31 倍高速化（倍精度）

SpTRSV の概要

- SpTRSV（線形方程式 $L\mathbf{x} = \mathbf{b}$ を解く）
  - L: 疎行列, 下三角
- 線形方程式の求解で利用
  - 反復法の前処理
  - マルチグリッド法の Smoothing と Correction
- 性能ボトルネックになりがち
- グリッド状モデルのシミュレーションで応用
  - Ex.) 電磁気学, 天体物理学, 放射輸送, 大気モデル
- 高速化の方針はおおまかに 2 種類
  - Level Scheduling: 同じ深さのノードの計算を並列実行
  - Reordering: 元の行列の頂点番号を付け替えて並列化



研究のモチベーション

- 既存研究では, ベクトル命令をあまり使えていない
- 行列の格納形式を工夫
- グリッドにおける計算の特性を利用
  - データ管理・アルゴリズム設計がらくちん
  - メモリ使用率が低い
  - 計算効率がよい
- 方針:
  - BCSR + DIA 形式
  - BMC にベクトル命令を適用できるようアレンジ

① SpTRSV とリオーダリング

② 格納形式と計算方法

③ DBSR 形式上での ILU(0)

④ 評価

⑤ まとめ

LU 分解

- 下（上）三角行列 L の性質：$L^{-1}\mathbf{s}$ を L^{-1} を構築せずに計算可！
- LU 分解：行列 A を，$A = LU$（下三角×上三角）に分解
  - Q. なにが嬉しい？
  - A. 方程式 $A\mathbf{x} = \mathbf{b}$ を，A^{-1} を構築せずに計算可！
    - 右辺ベクトルを変えた方程式を複数解くときに有効
- 算法：

  $\forall(i,j)$ に対して $a_{ij} = a_{ij} - \sum_k a_{ik}a_{kj}$

$$\because A\mathbf{x} = \mathbf{b}$$
$$\Leftrightarrow LU\mathbf{x} = \mathbf{b}$$
$$\Leftrightarrow \mathbf{x} = U^{-1}(L^{-1}\mathbf{b})$$

ILU(0) 分解

- LU 分解の欠点
  - 時間計算量 $O(n^3)$
  - 行列 A に比べ，L, U が密になる
- ILU(0) 分解：行列 A のスパースパターン S を保ち，$A \approx LU$ に
- 算法：

$a_{ij} \neq 0$ なる (i, j) に対して $a_{ij} = a_{ij} - \sum_k a_{ik} a_{kj}$

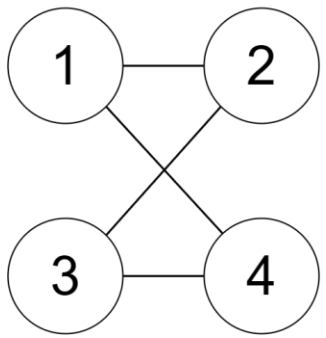
精度を犠牲に計算コストを低減

ILU(0) と反復法の関係

- 反復法は係数行列 A が単位行列に近いほど収束性 UP
- 前処理：ILU(0) などにより $A\mathbf{x} = \mathbf{b}$ を $(LU)^{-1}A\mathbf{x} = (LU)^{-1}\mathbf{b}$ として変形
  - L や U を前処理行列と呼ぶ
  - $A \approx LU$ より，$(LU)^{-1}A \approx I$
  - 反復毎の処理量は増えるが，収束性が大きく向上
  - $(LU)^{-1}A$ は陽に計算せず，反復毎に SpTRSV を介し同値な操作を行う

SpTRSV の流れ

- 以下の行列 A を ILU(0) 分解．このときの L の SpTRSV を考える



$$\begin{bmatrix} a_{1,1} & a_{1,2} & & a_{1,4} \\ a_{2,1} & a_{2,2} & a_{2,3} & \\ & a_{3,2} & a_{3,3} & a_{3,4} \\ a_{4,1} & & a_{4,3} & a_{4,4} \end{bmatrix} \xrightarrow{\text{ILU(0)}} \begin{bmatrix} l_{1,1} & & & \\ l_{2,1} & l_{2,2} & & \\ & l_{3,2} & l_{3,3} & \\ l_{4,1} & & l_{4,3} & l_{4,4} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix}$$

- $x_1 = \dfrac{b_1}{l_{1,1}}$
- $x_2 = \dfrac{b_2 - l_{2,1}x_1}{l_{2,2}}$
- $x_3 = \dfrac{b_3 - a_{3,2}x_2}{a_{3,3}}$
- $x_4 = \dfrac{b_4 - a_{4,1}x_1 - a_{4,3}x_3}{a_{3,3}}$

掃き出し法のように
上の行から順に求める

SpTRSV におけるデータの依存関係

- データの依存関係のため，並列化は難しい

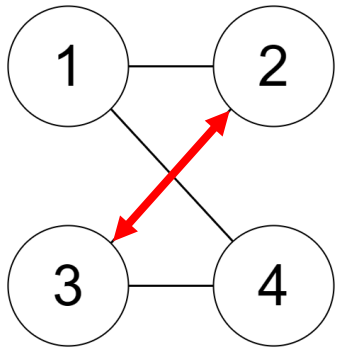
$$\begin{bmatrix} l_{1,1} & & & \\ l_{2,1} & l_{2,2} & & \\ & l_{3,2} & l_{3,3} & \\ l_{4,1} & & l_{4,3} & l_{4,4} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix}$$

- $x_1 = \dfrac{b_1}{l_{1,1}}$
- $x_2 = \dfrac{b_2 - l_{2,1} x_1}{l_{2,2}}$
- $x_3 = \dfrac{b_3 - a_{3,2} x_2}{a_{3,3}}$
- $x_4 = \dfrac{b_4 - a_{4,1} x_1 - a_{4,3} x_3}{a_{3,3}}$

前の計算結果が必要
どう並列化するか？

依存関係の変更

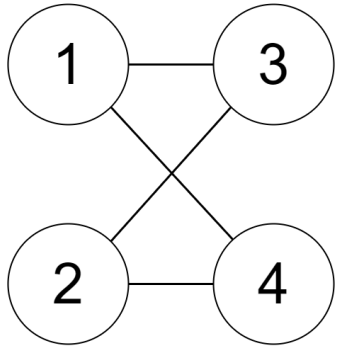
- リオーダリング: 行列 A の頂点番号を振り直して, 依存関係を変える



$a_{1,1}$	$a_{1,2}$		$a_{1,4}$
$a_{2,1}$	$a_{2,2}$	$a_{2,3}$	
	$a_{3,2}$	$a_{3,3}$	$a_{3,4}$
$a_{4,1}$		$a_{4,3}$	$a_{4,4}$

ILU(0)

$l_{1,1}$			
$l_{2,1}$	$l_{2,2}$		
	$l_{3,2}$	$l_{3,3}$	
$l_{4,1}$		$l_{4,3}$	$l_{4,4}$

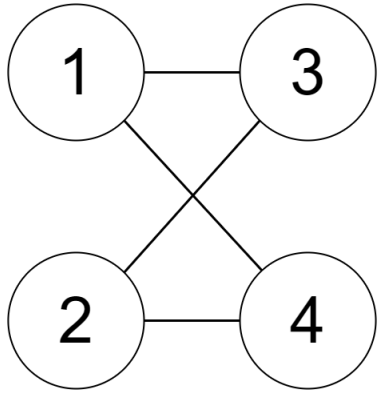


$a_{1,1}$		$a_{1,3}$	$a_{1,4}$
	$a_{2,2}$	$a_{2,3}$	$a_{2,4}$
$a_{3,1}$	$a_{3,2}$	$a_{3,3}$	
$a_{4,1}$	$a_{4,2}$		$a_{4,4}$

ILU(0)

$l_{1,1}$			
	$l_{2,2}$		
$l_{3,1}$	$l_{3,2}$	$l_{3,3}$	
$l_{4,1}$	$l_{4,2}$		$l_{4,4}$

リオーダリングによる SpTRSV の並列化



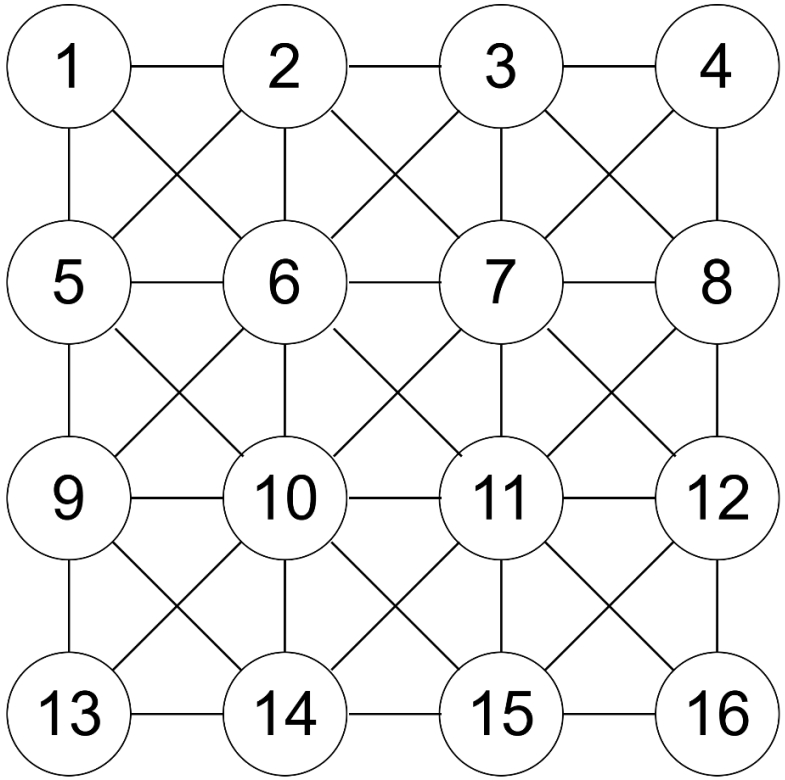
$$
\begin{bmatrix} a_{1,1} & & a_{1,3} & a_{1,4} \\ & a_{2,2} & a_{2,3} & a_{2,4} \\ a_{3,1} & a_{3,2} & a_{3,3} & \\ a_{4,1} & a_{4,2} & & a_{4,4} \end{bmatrix}
\xrightarrow{\text{ILU(0)}}
\begin{bmatrix} l_{1,1} & & & \\ & l_{2,2} & & \\ l_{3,1} & l_{3,2} & l_{3,3} & \\ l_{4,1} & l_{4,2} & & l_{4,4} \end{bmatrix}
\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}
=
\begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix}
$$

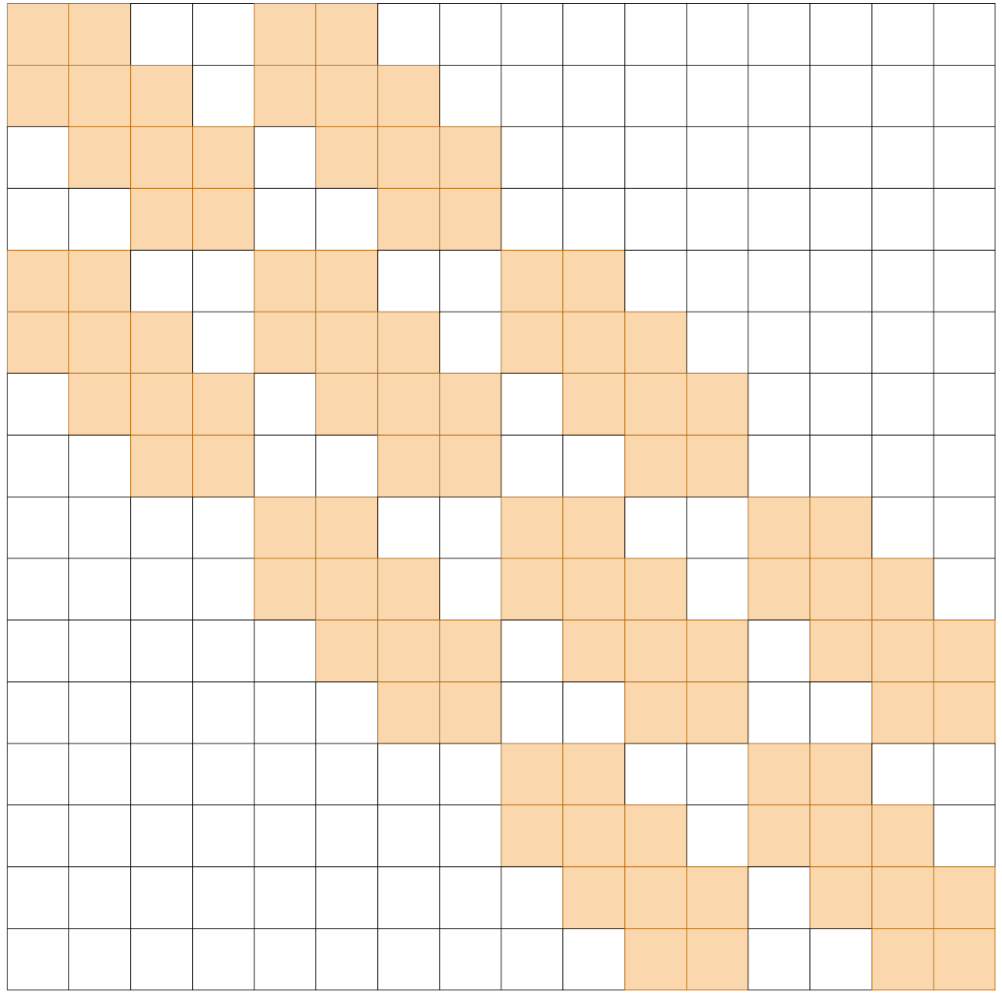
- $x_1 = \dfrac{b_1}{l_{1,1}}$
- $x_2 = \dfrac{b_2}{l_{2,2}}$

- $x_3 = \dfrac{b_3 - l_{3,1}x_1 - l_{3,2}x_2}{l_{3,3}}$
- $x_4 = \dfrac{b_4 - l_{4,1}x_1 - l_{4,2}x_2}{l_{4,4}}$

赤枠内で
並列化できた！

どのように頂点番号を振り直せば依存関係を緩和できる？

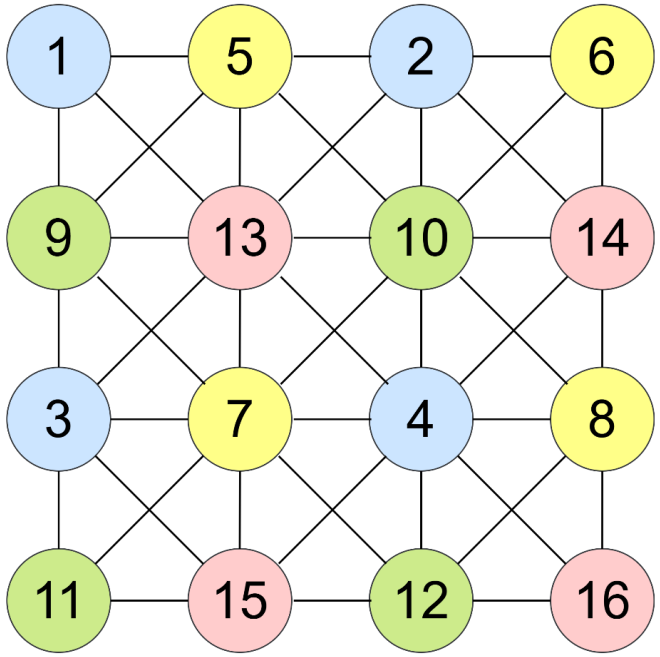


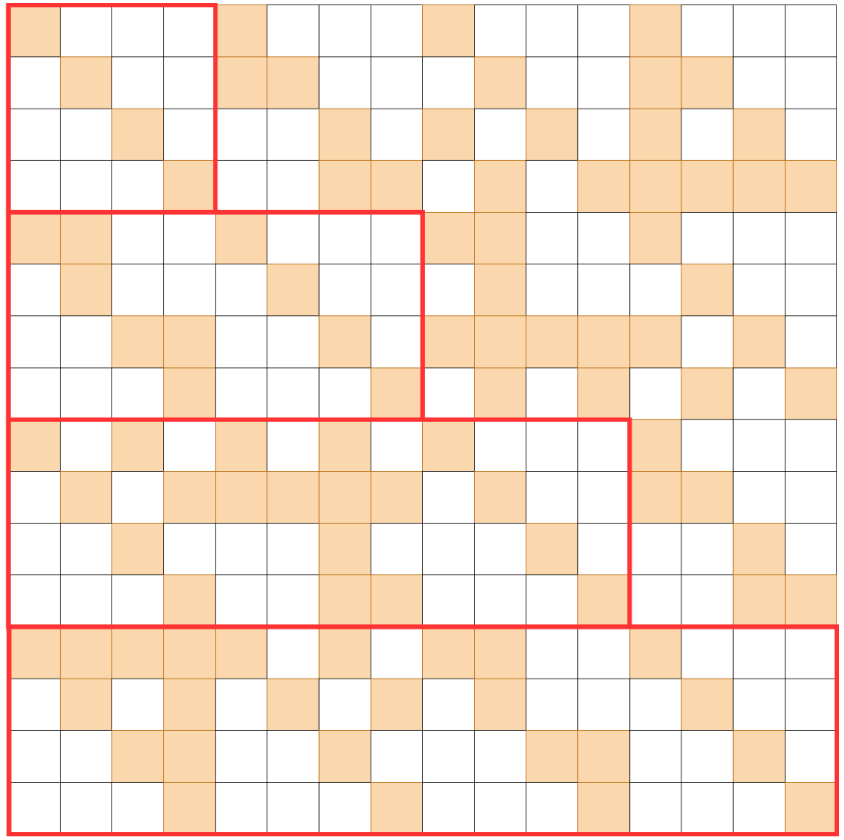


マルチカラーオーダリング

- マルチカラーオーダリング（MC）：
  1. グラフを k-彩色
  2. 同色の頂点に連番で番号付け

同色の頂点どうしは
計算順序に依存性がない



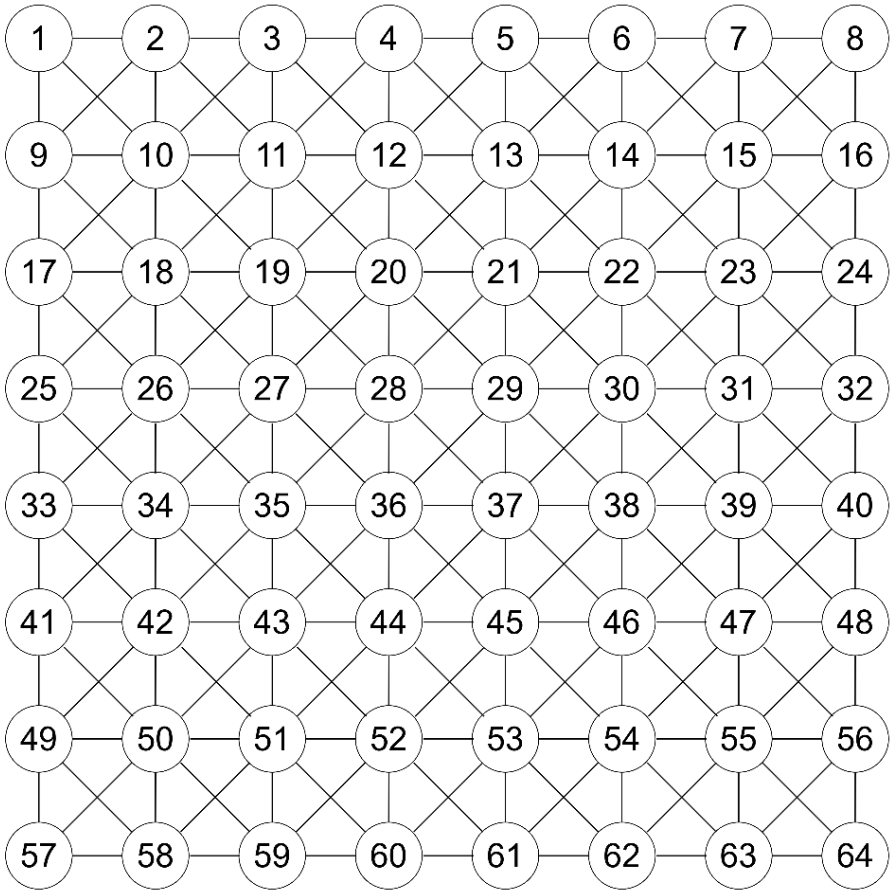


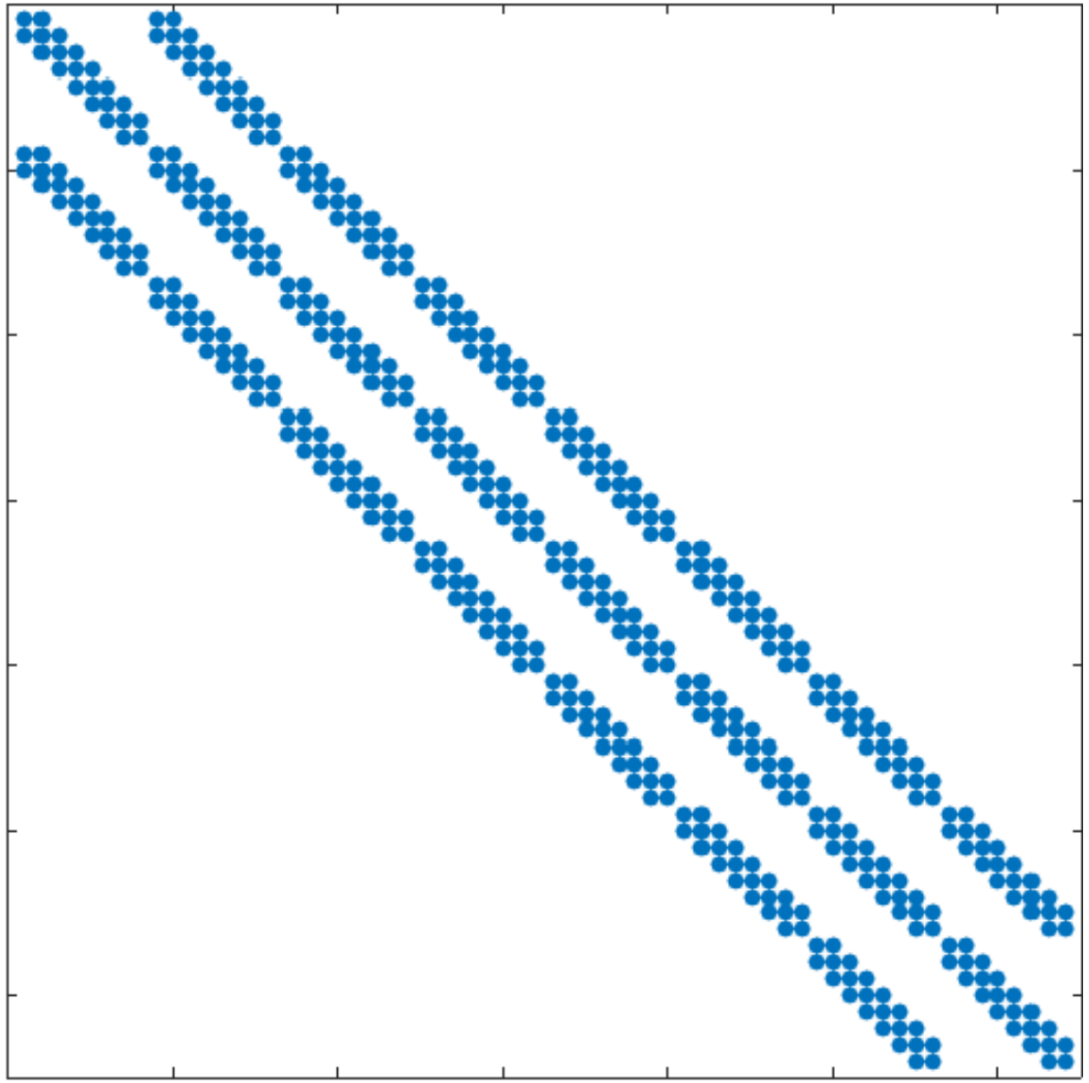
MC の問題点

- 並列性と収束性はトレードオフの傾向
- 色数 k の決定が難しい
  - k が小さすぎ：破綻（or そもそも彩色不可能）
    - Tips: 判定問題「一般グラフが k-彩色可能か？」は NP 完全 😢
  - k が大きすぎ：参照局所性が低下, 同期コスト大

高速化典型：
ブロック化して改善！

MC をブロック化しつつ, 依存関係を緩和したい.





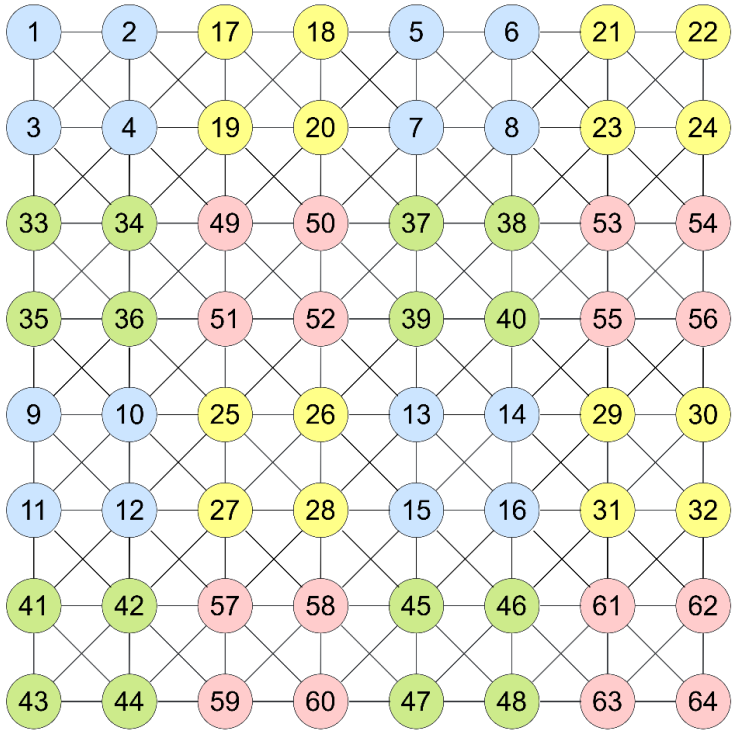
ブロックマルチカラーオーダリング

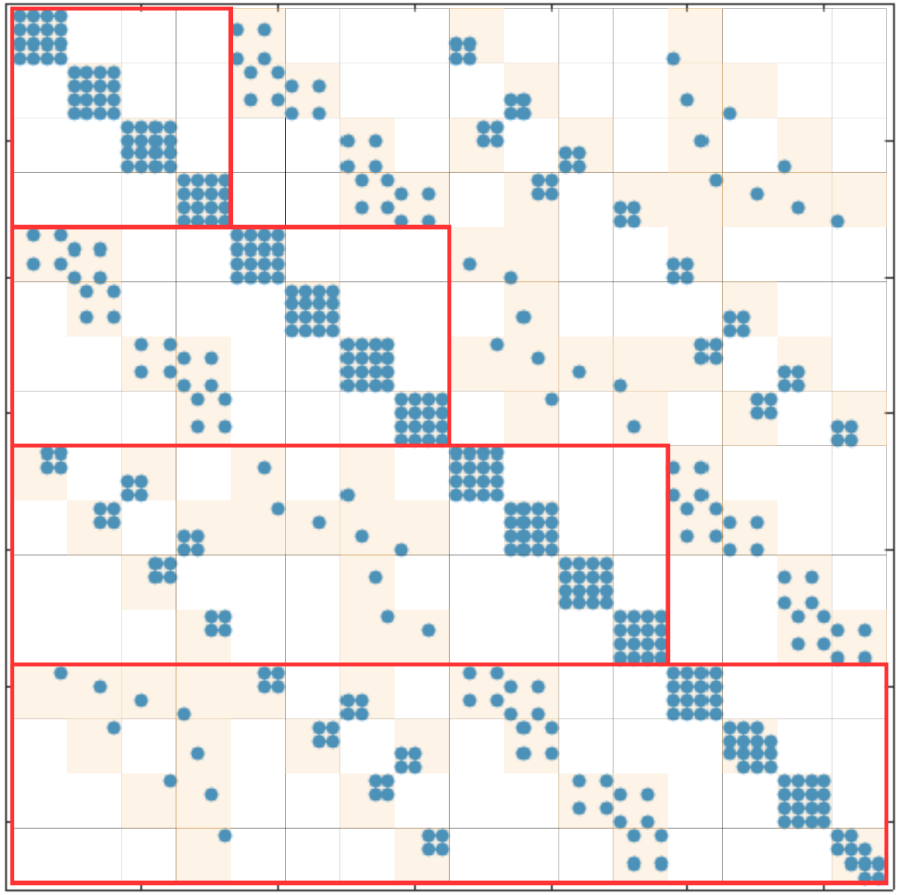
- ブロックマルチカラーオーダリング（BMC）：
  1. 隣接頂点をブロックとして縮約
  2. 縮約したグラフを MC
  3. MC したグラフを拡逆
  4. ブロック内の頂点に連番で番号付け

同色のブロックどうしは
計算順序に依存性がない

グラフの縮約で
彩色が簡単に

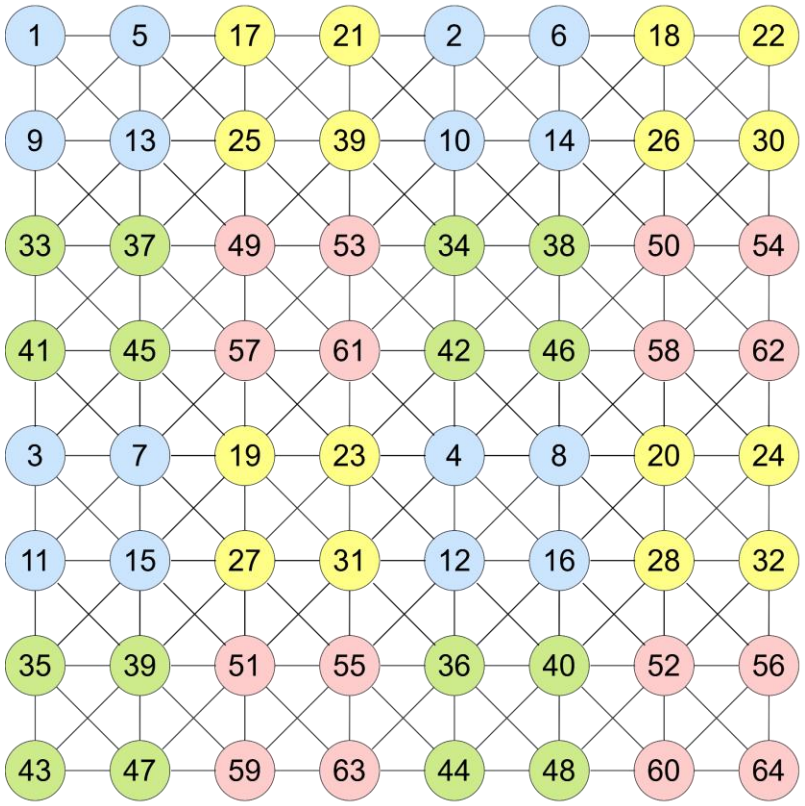
ブロック化で
局所性が向上

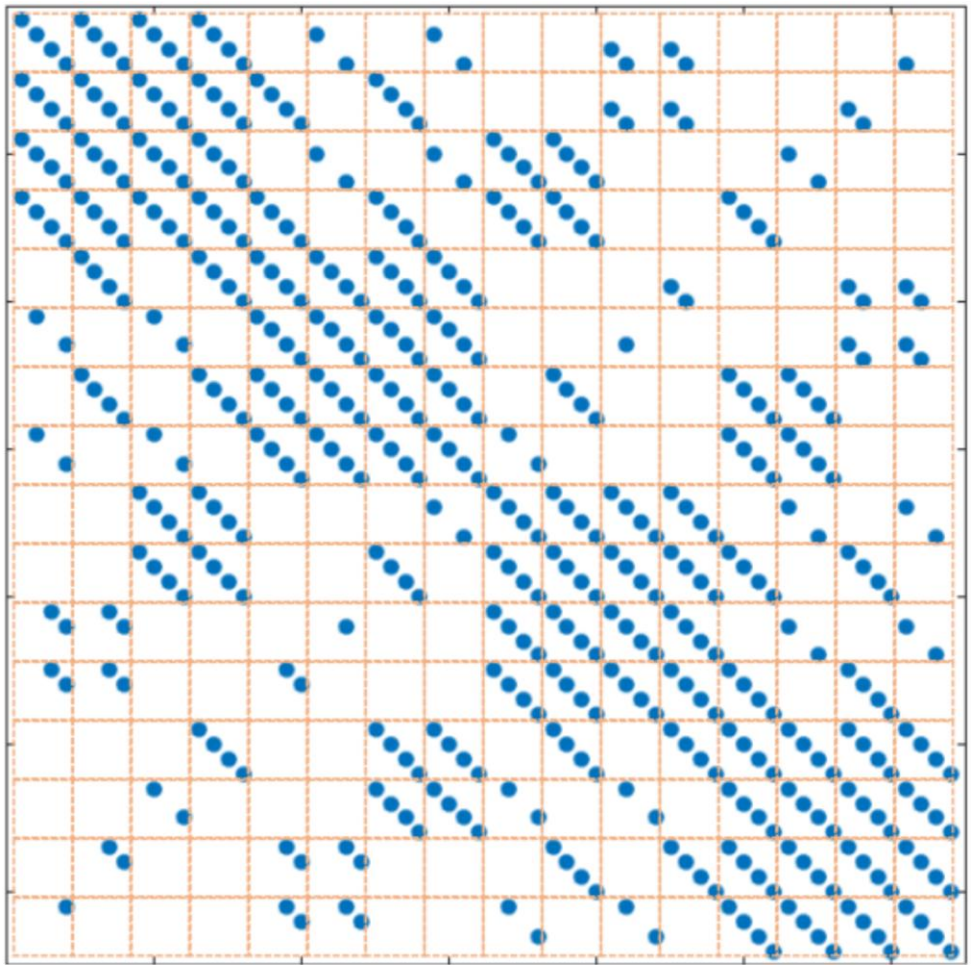




【提案手法】BMC を SIMD 化する

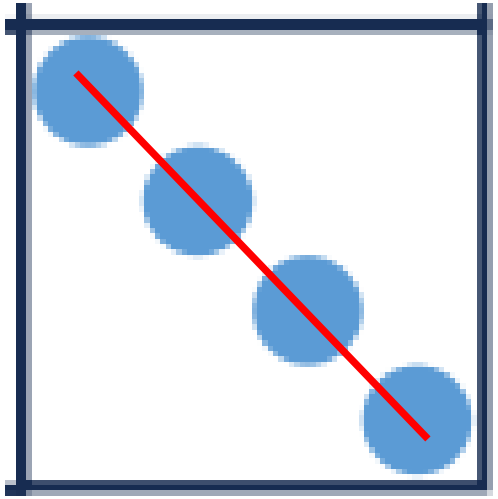
- Vectorized BMC（VBMC）:
  1. グラフを縮約, MC, 拡逆
  2. 各ブロックの同じ場所にある頂点に連番

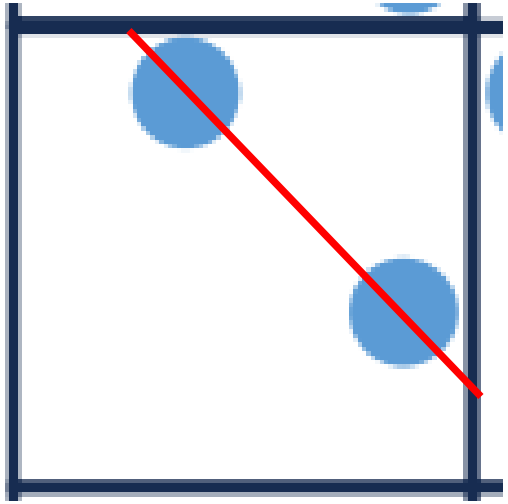




VBMC の特徴

- 斜めに要素が詰まったブロックで構成
- ベクトル命令でまとめて処理
- 接続関係が不規則なモデルには適用不可
  - グラフ同型なブロックを複数作れることが前提
- 具体的な格納形式や計算方法は後述





① SpTRSV とリオーダリング

② 格納形式と計算方法

③ DBSR 形式上での ILU(0)

④ 評価

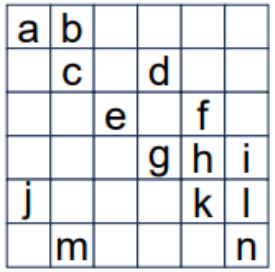
⑤ まとめ

疎行列の格納形式

- 疎行列を $n \times n$ 変数で記録するのは非効率
  - 一般的に, 非ゼロ要素数は $O(n)$ 個
  - 効率的に非ゼロ要素にアクセスしたい
- さまざまな形式が提案されている
  - Ex.) COO, CSR, CSC, DIA, SELL-C-σ, BCSR, JAD, VBR...
- キャッシュ効率, 計算速度に大きな影響
- VBMC と相性の良い格納形式を作りたい

早見表

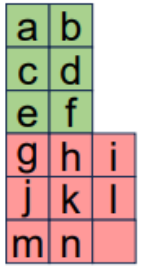
- CSR
  - 行ごとに非ゼロ要素を圧縮
  - 各行の先頭要素へのポインタ



row_ptr = [0, 2, 4, 6, 9, 12, 14]
col_ind = [0,1,1,3,2,4,3,4,5,0,4,5,1,5]
values = [a,b, c,d, e,f, g,h,i, j,k,l, m,n]

(a) CSR

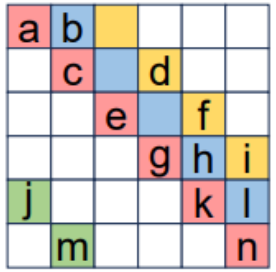
- Sliced ELLPACK (SELL)
  - 行をいくつかのスライスに分割
  - 各スライスの最大要素数までゼロ埋め



slice_offset = [0, 6, 15]
col_ind = [0,1,2,1,3,4,3,0,1,4,4,5,5,5,0]
values = [a,c,e, b,d,f, g,j,m, h,k,n, i,l,0]

(b) SELL

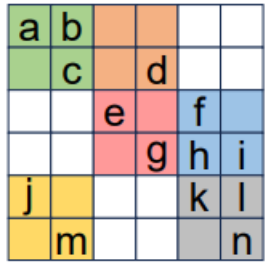
- Diagonal (DIA)
  - 対角線方向に格納
  - 要素がなければゼロ埋め



offset = [-4, 0, 1, 2]
values = [0,0,0,0,j,m; a,c,e,g,k,n;
b,0,0,h,l,0; 0,d,f,i,0,0]

(c) DIA

- Block Compressed Sparse Row (BCSR)
  - 行列をブロック分割
  - 各ブロックへのポインタと列インデクス

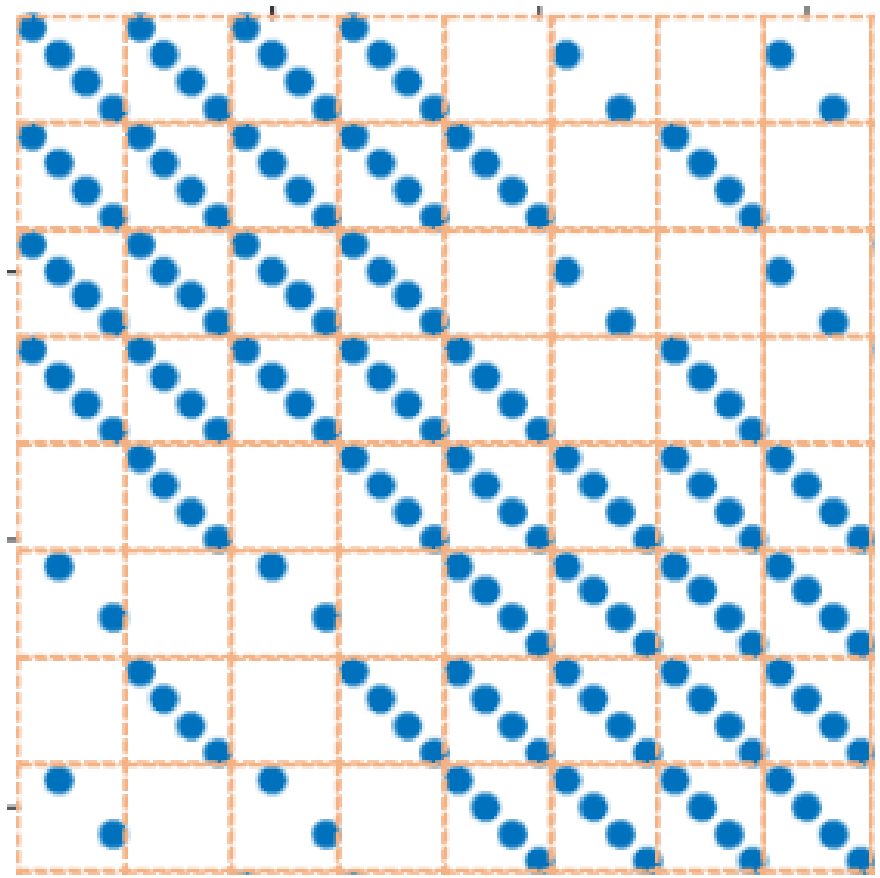


bsize = 2
blk_ptr = [0, 2, 4, 6]
blk_ind = [0, 1, 1, 2, 0, 2]
values = [a,0,b,c; 0,0,0,d; e,0,0,g;
f,h,0,i; j,0,0,m; k,0,l,n]

(d) BCSR

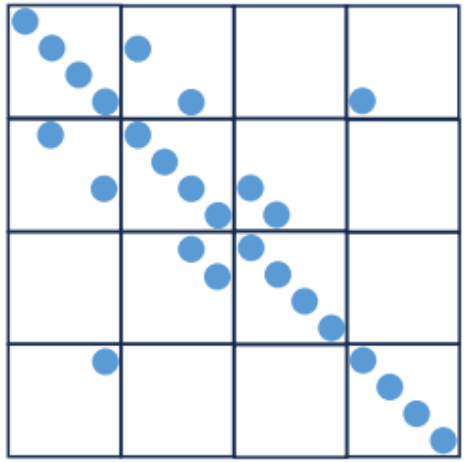
VBMC 向けの格納形式の設計

- 各ブロックに, 斜めに非ゼロ要素がある行列
- アイデア: BCSR + DIA
  - 各ブロックの位置を CSR-like に管理
  - ブロック内の非ゼロ要素を DIA で保存



【提案手法】Diagonal Block CSR

- Diagonal Block CSR（DBSR）
  - brow: ブロックが何行あるか
  - bsize: ブロックサイズ（注：≠縮約グラフのブロックサイズ）
    - bsize=1: 通常のBMC
  - blk_ptr[r]: r 行目左端のブロックへのポインタ
  - blk_ind[i]: i 個目のブロックの列インデクス
  - blk_offset[i]: i 個目のブロック内において，非ゼロ要素が並ぶ斜めのラインがどの位置か
    - ブロックの対角線に重なるなら 0
  - values[i]: i 個目の非ゼロ要素
- 各ブロックには必ず brow 個の非ゼロ要素を格納
  - なければゼロ埋め
- CSR よりも列ポインタと行インデクスが軽量



brow = 4

bsize = 4

blk_ptr = [0,3,6,8,10]

blk_ind = [0,1,3,0,1,2,
1,2,0,3]

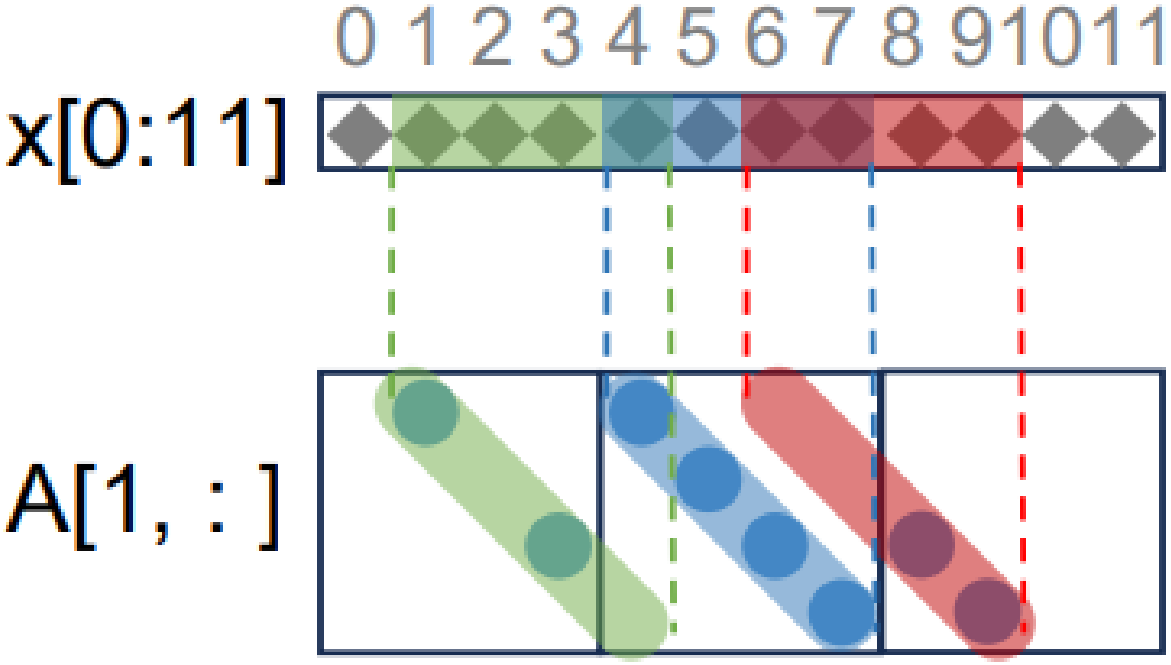
blk_offset = [0,-1,-3,1,0,-2,
-2,0,3,0]

values = [*,*,*,*; 0,*,0,*; 0,0,0,*;
,0,,0; *,*,*,*; 0,0,*,*; *,*,0,0;
,,*,*; *,0,0,0; *,*,*,*]

gather-free な更新

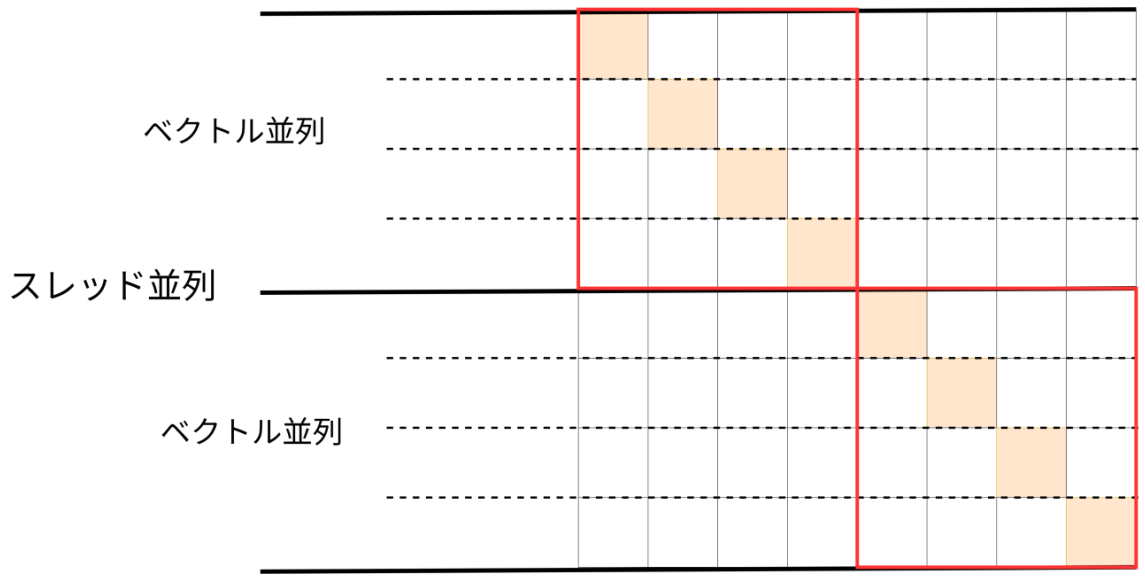
- ベクトル計算におけるロードの種類は主に 2 つ
  - vload: 連続したアドレスからロードする. 高速
  - gather: 連続しないアドレスからロードする. 低速
- メモリ律速なベクトル計算は, いかに gather を vload にできるかが重要

DBSR の SpTRSV は
x からのロードが
連続になっている!



DBSR を用いた SpTRSV

- 左上のブロックから順に更新
- ブロックをスレッド並列
- ブロック内をベクトル並列



Input: L, b, x ▷ L is the lower triangular matrix.
Output: x

```
// for color from 0 to NumColors − 1
// #pragma omp parallel for
// for t from BlocksNum[color] to BlocksNum[color + 1] − 1
for i = t * BlockSize to (t + 1) * BlockSize − 1 do
    vdouble vec_temp = load(b + i * bsize)
    for j = blk_ptr[i] to blk_ptr[i + 1] − 1 do
        int ind_val = j * bsize
        int ind_x = blk_ind[j] * bsize + blk_offset[j]
        vdouble vec_vals = load(values + ind_val)
        vdouble vec_x = load(x + ind_x)
        vec_temp− = vec_vals * vec_x
    end
    vec_store(x + i * bsize, vec_temp)
end
```

① SpTRSV とリオーダリング
② 格納形式と計算方法
③ DBSR 形式上での ILU(0)
④ 評価
⑤ まとめ

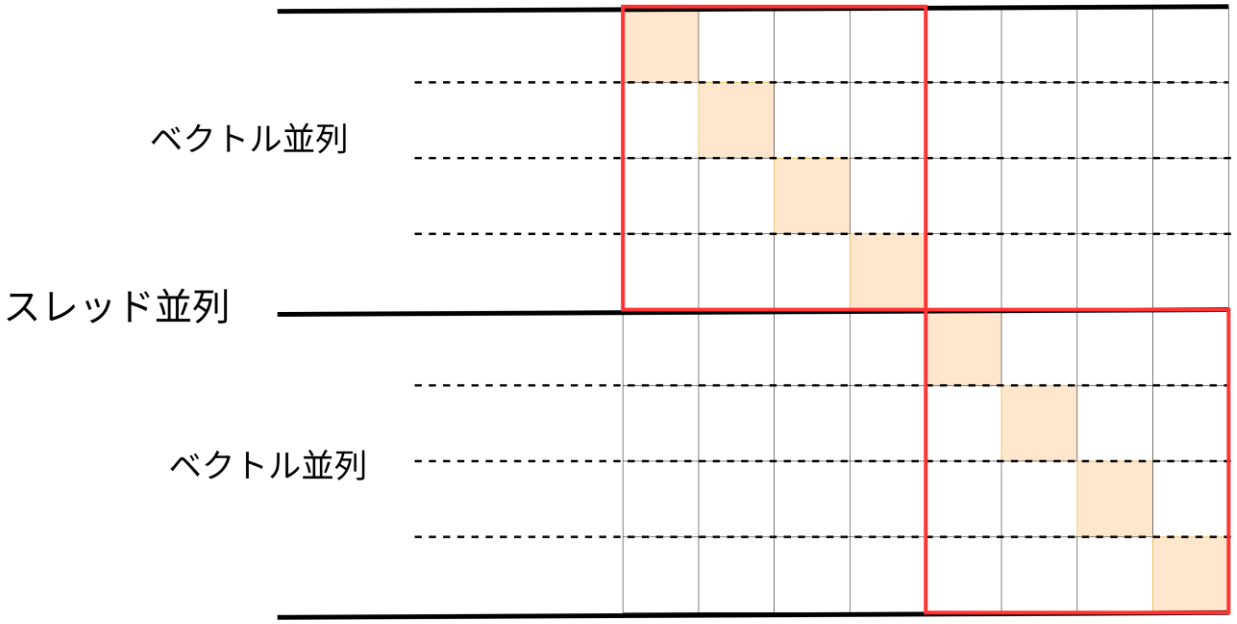
ILU(0) の算法

- ILU(0) は以下の手続きによって計算される
- 下三角部分が L, 上三角部分が U に
- データの依存関係のため, 並列化は難しい

```
for i = 1 to n − 1 do
    for k = 0 to i − 1 and (i, k) ∈ NZ(A) do
        Compute a_ik = a_ik / a_kk
        for j = k + 1 to n − 1 and (i, j) ∈ NZ(A) do
            Compute a_ij := a_ij − a_ik a_kj.
        end
    end
end
```

ILU(0) の並列化

- 実は, VBMC によって ILU(0) も並列化可能に
- DBSR 形式のもとで, SpTRSV と同様に並列化
  - ブロックをスレッド並列
  - ブロック内をベクトル並列



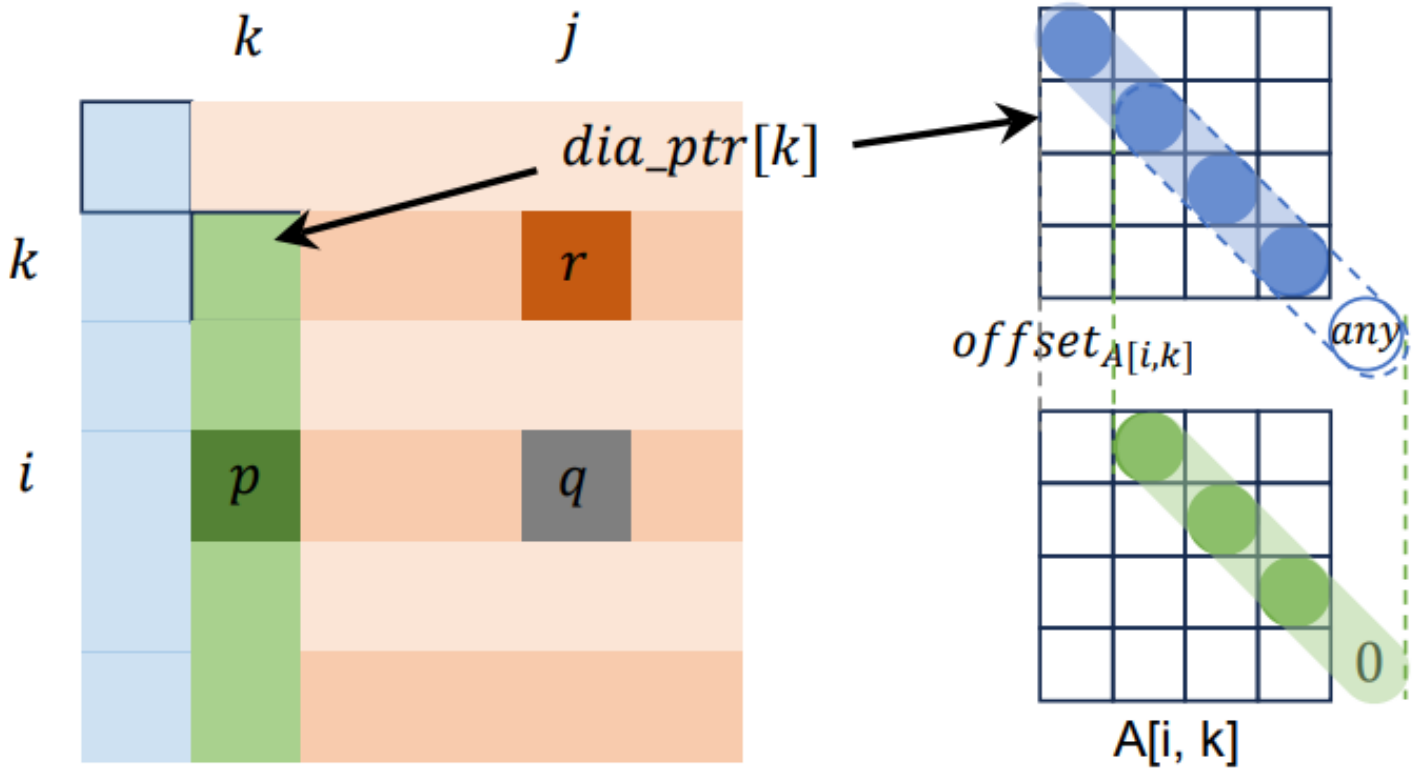

```
// for color from 0 to NumColors − 1
// #pragma omp parallel for
// for t from BlocksNum[color] to BlocksNum[color + 1] − 1
for i = t * BlockSize + 1 to (t + 1) * BlockSize − 1 do
    for p = blk_ptr[i] to dia_ptr[i] − 1 do
        int k = blk_ind[p]
        vdouble vec_A_ik = load(values + p * bsize)
        vdouble vec_A_kk =
          load(values + dia_ptr[k] * bsize + blk_offset[p])
        vec_A_ik = vec_A_ik / vec_A_kk
        while r = dia_ptr[k] + 1 to blk_ptr[k + 1] − 1 and
          q = p + 1 to blk_ptr[i + 1] − 1 do
            if blk_ind[r] == blk_ind[q] and
              blk_offset[p] + blk_offset[r] == blk_offset[q]
              then
                vdouble vec_A_ij = load(values + q * bsize)
                vdouble vec_A_kj =
                  load(values + r * bsize + blk_offset[p])
                vec_A_ij − = vec_A_ik * vec_A_kj
            end
        end
    end
end
```

ILU(0) における除算処理のベクトル化

- 対角成分で除算する部分の処理
- ゼロ埋め部分に対応する対角成分はなんでもよい
  - あえて範囲外アクセス(?)してロードできる

```
for i = 1 to n − 1 do
    for k = 0 to i − 1 and (i, k) ∈ NZ(A) do
        Compute a_ik = a_ik / a_kk
        for j = k + 1 to n − 1 and (i, j) ∈ NZ(A) do
            Compute a_ij := a_ij − a_ik a_kj .
        end
    end
end
```

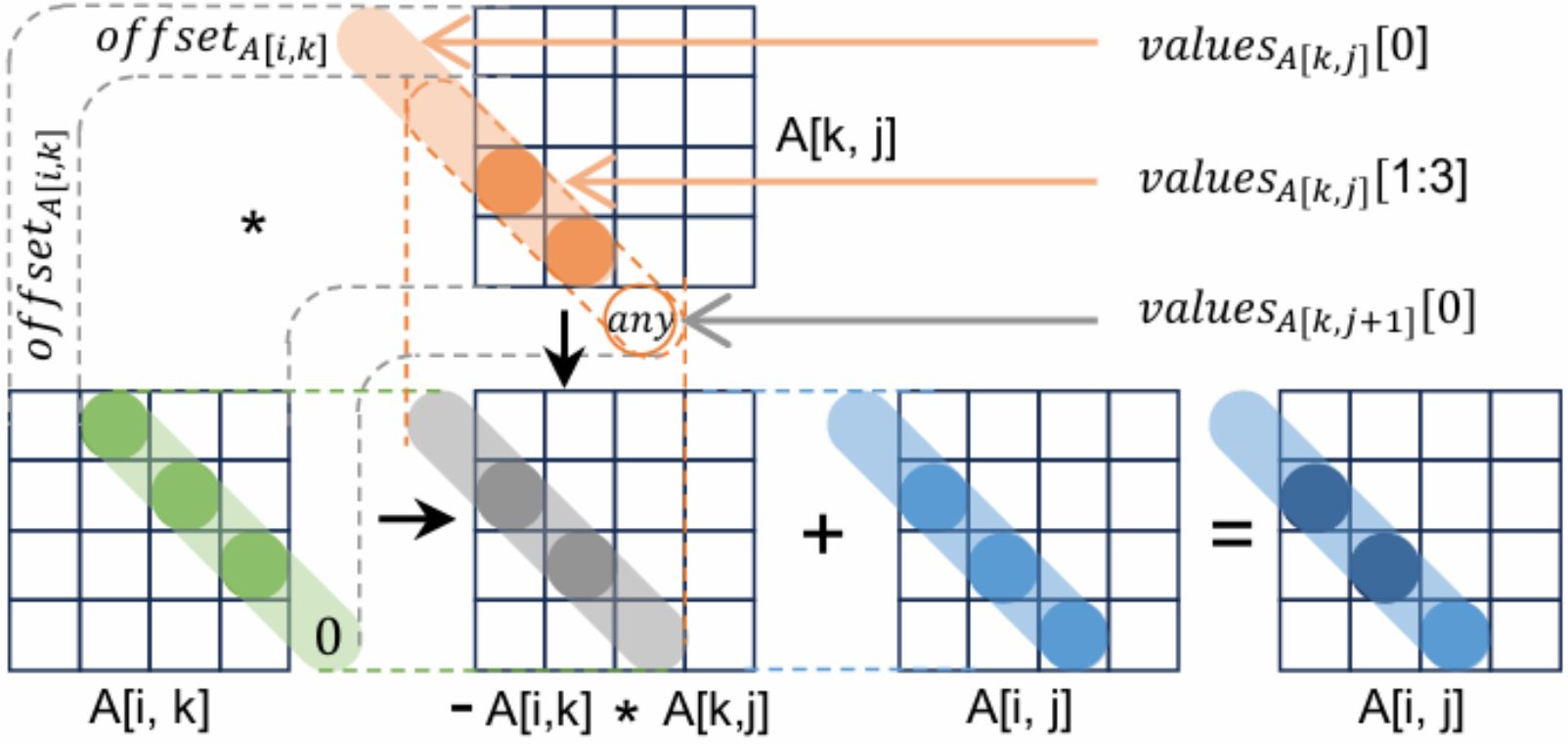
```
// for color from 0 to NumColors − 1
// #pragma omp parallel for
// for t from BlocksNum[color] to BlocksNum[color + 1] − 1
for i = t * BlockSize + 1 to (t + 1) * BlockSize − 1 do
    for p = blk_ptr[i] to dia_ptr[i] − 1 do
        int k = blk_ind[p]
        vdouble vec_A_ik = load(values + p * bsize)
        vdouble vec_A_kk =
          load(values + dia_ptr[k] * bsize + blk_offset[p])
        vec_A_ik = vec_A_ik / vec_A_kk
        while r = dia_ptr[k] + 1 to blk_ptr[k + 1] − 1 and
          q = p + 1 to blk_ptr[i + 1] − 1 do
            if blk_ind[r] == blk_ind[q] and
              blk_offset[p] + blk_offset[r] == blk_offset[q]
              then
                vdouble vec_A_ij = load(values + q * bsize)
                vdouble vec_A_kj =
                  load(values + r * bsize + blk_offset[p])
                vec_A_ij − = vec_A_ik * vec_A_kj
            end
        end
    end
end
```



ILU(0) における乗算・減算処理のベクトル化

- 乗算・減算する部分の処理
- ゼロ埋め部分に対応する対角成分はなんでもよい
  - あえて範囲外アクセス(?)してロードできる

```
for i = 1 to n − 1 do
    for k = 0 to i − 1 and (i, k) ∈ NZ(A) do
        Compute a_ik = a_ik / a_kk
        for j = k + 1 to n − 1 and (i, j) ∈ NZ(A) do
            Compute a_ij := a_ij − a_ik a_kj.
        end
    end
end
```




```
// for color from 0 to NumColors − 1
// #pragma omp parallel for
// for t from BlocksNum[color] to BlocksNum[color + 1] − 1
for i = t * BlockSize + 1 to (t + 1) * BlockSize − 1 do
    for p = blk_ptr[i] to dia_ptr[i] − 1 do
        int k = blk_ind[p]
        vdouble vec_A_ik = load(values + p * bsize)
        vdouble vec_A_kk =
          load(values + dia_ptr[k] * bsize + blk_offset[p])
        vec_A_ik = vec_A_ik / vec_A_kk
        while r = dia_ptr[k] + 1 to blk_ptr[k + 1] − 1 and
          q = p + 1 to blk_ptr[i + 1] − 1 do
            if blk_ind[r] == blk_ind[q] and
              blk_offset[p] + blk_offset[r] == blk_offset[q]
              then
                vdouble vec_A_ij = load(values + q * bsize)
                vdouble vec_A_kj =
                  load(values + r * bsize + blk_offset[p])
                vec_A_ij −= vec_A_ik * vec_A_kj
            end
        end
    end
end
```

① SpTRSV とリオーダリング
② 格納形式と計算方法
③ DBSR 形式上での ILU(0)

④ 評価

⑤ まとめ

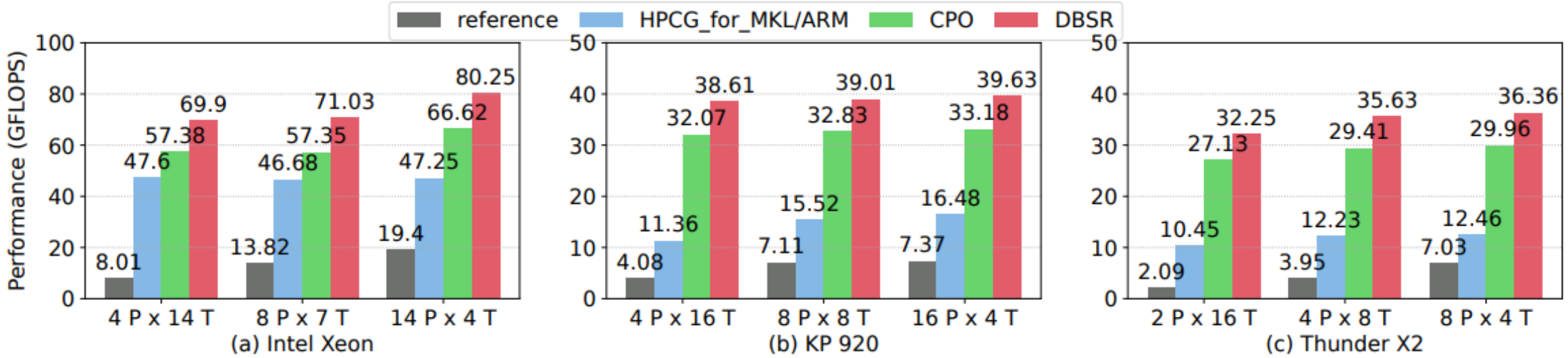
セットアップ

- Intel, ARM の両方
- DBSR の性能検証は, HPCG ベンチマークに基づいて実験
  - nx = ny = nz = 192 の 3 次元グリッド
- ILU(0) の性能検証は, nx = ny = nz = 256 のステンシル計算で実験
- レーン幅: double × 8

	Intel Xeon	KP 920	Thunder X2	Phytium 2000+
Sockets	2	1	1	8
#Cores	2 × 28	1 × 64	1 × 32	1 × 64
#NUMAs	2	2	1	8
CPU Freq.	2.6GHz	2.6GHz	2.5GHz	2.2GHz
L1 Cache	80KB	64KB	32KB	32KB
L2 cache	1.25MB	512KB	256KB	2MB
L3 cache	42MB	64MB	32MB	None
SIMD-bits	AVX512-512	NEON-128	NEON-128	NEON-128

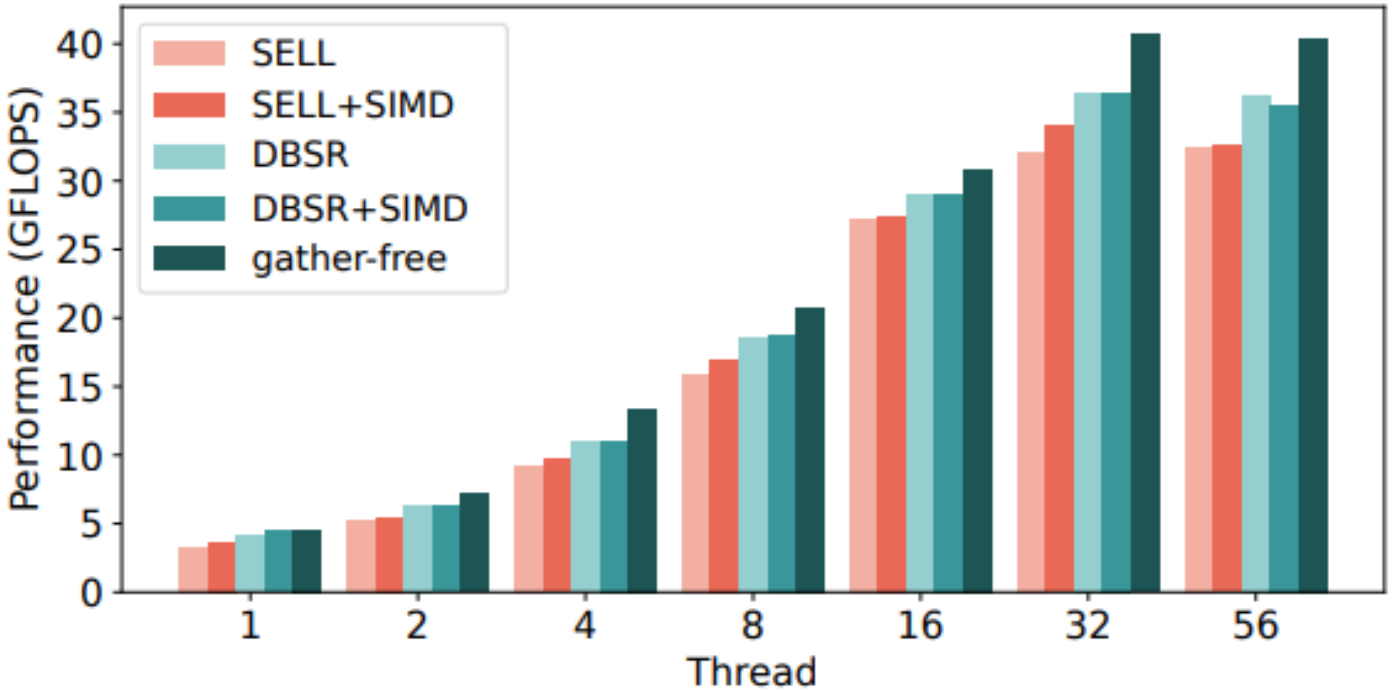
HPCG ベンチマークでの比較（計算性能）

- シングルノードでの比較
  - P: MPI Processes, T: OpenMP Threads
  - reference: 公式実装 HPCG ベンチマーク ver 3.1
  - CPO: いろいろ最適化された実装
- どのプロセス数, スレッド数においても計算性能が高い ！
  - 最大 3.40x（HPCG_for_ARM 比）



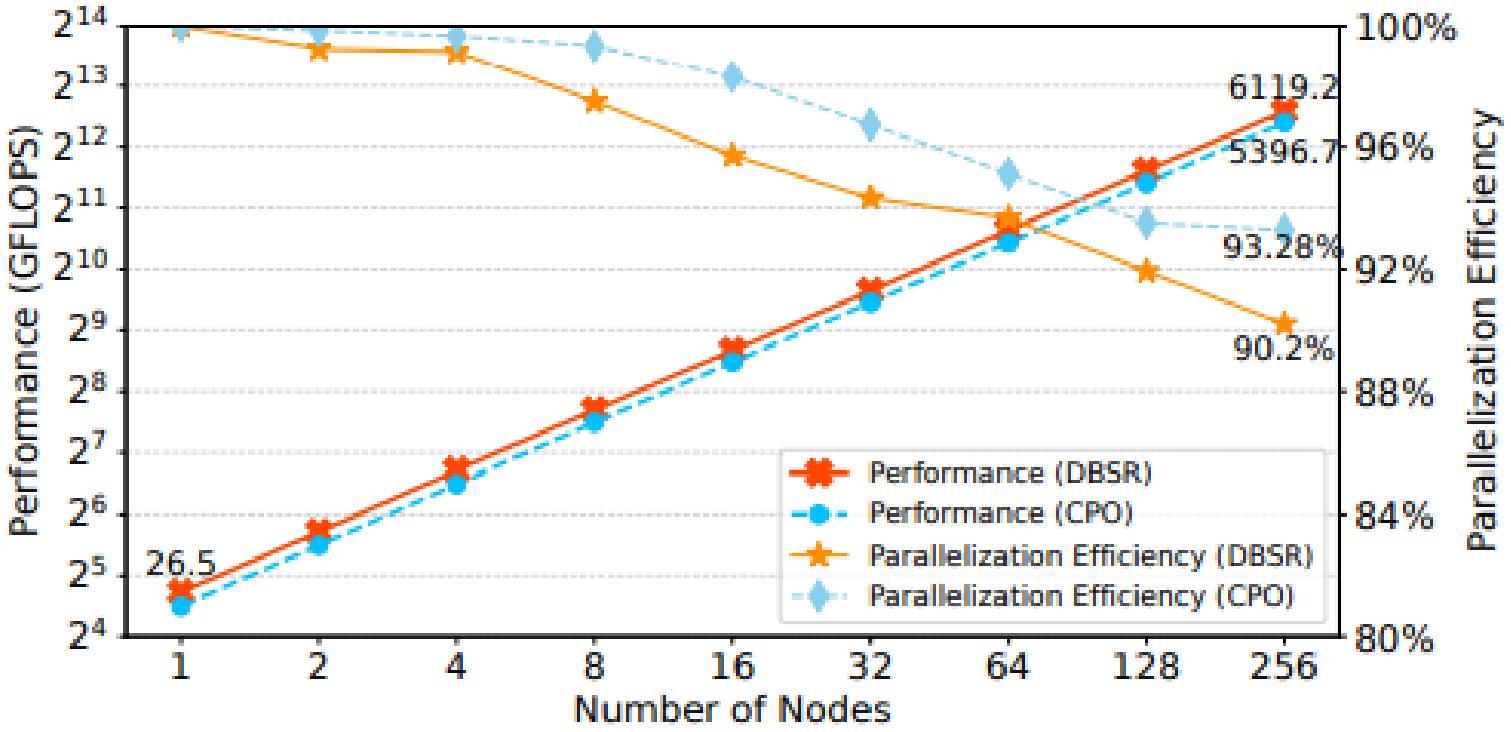
HPCG ベンチマークでの比較（SIMD の効果）

- SELL（SIMD あり／なし） v.s.
  DBSR（あえて gather を使った SIMD あり, SIMD あり, なり）
- gather-free の効果が大きいことが分かる
- 最大 1.16x （SELL 比）



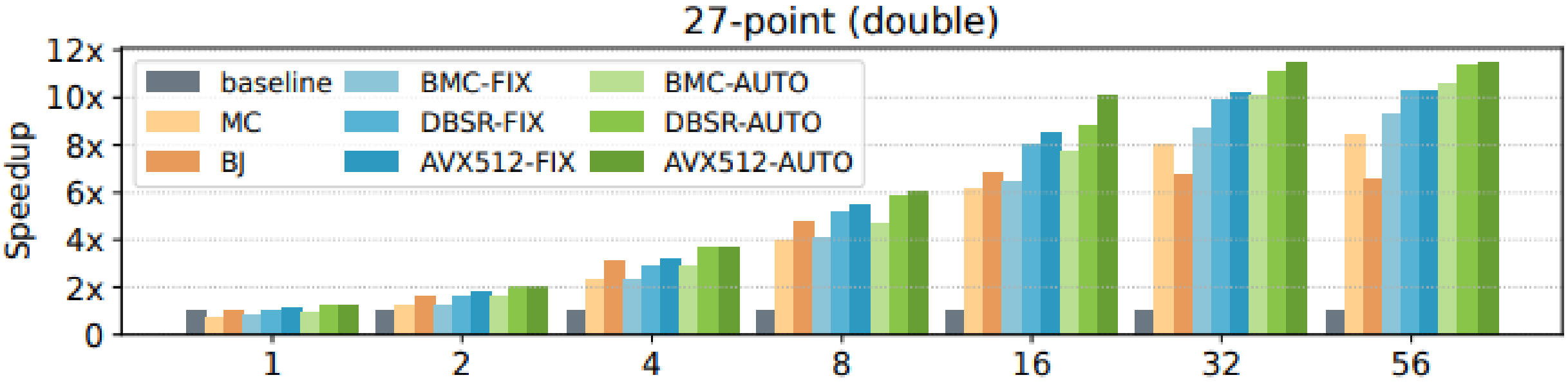
HPCG ベンチマークでの DBSR の弱スケーリング性

- 256 ノードの Phytium 2000+ クラスター上で、合計 16,384 コアを使用
- HPCG ベンチマークで弱スケーリング性を検証
- DBSR は最大 6,119 GFLOPS を達成（CPO 比 13.3% 改善）
- CPO に並列化効率は劣るが，90%↑をキープ



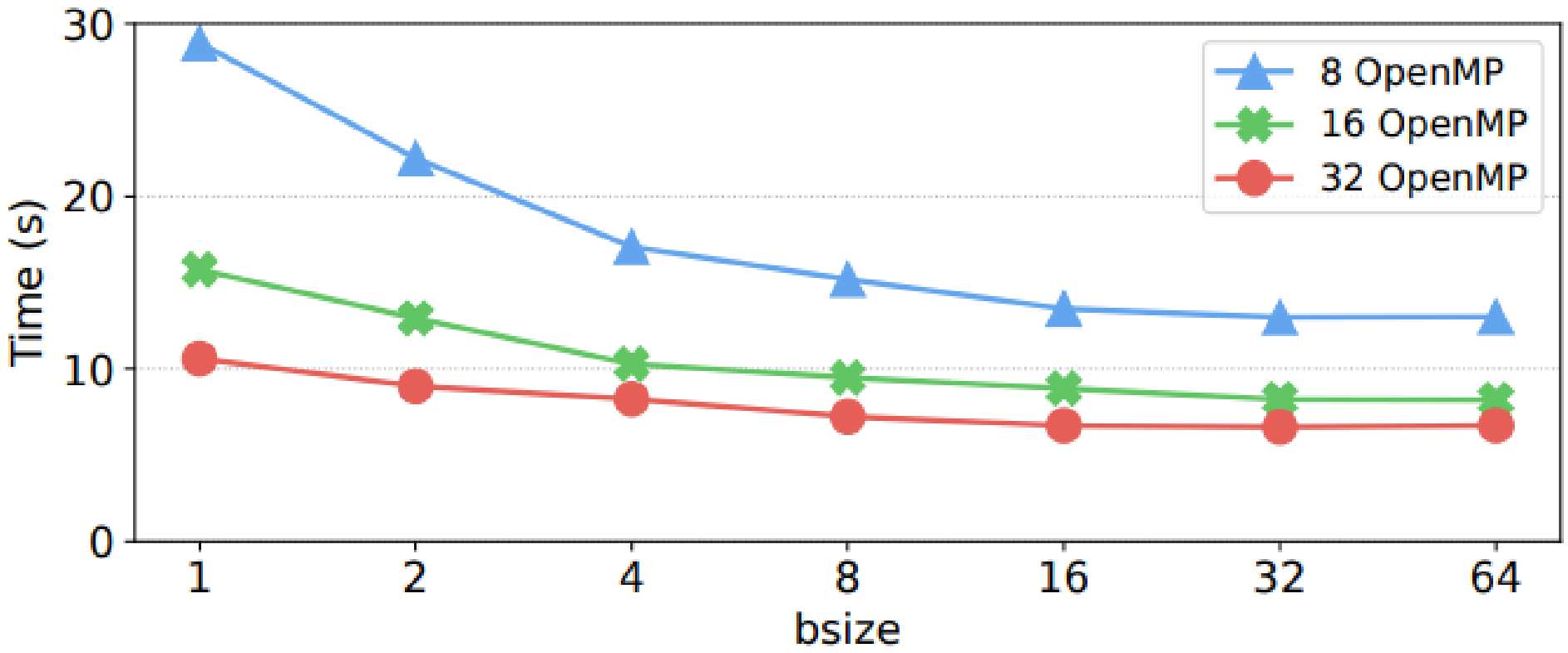
ILU(0) の高速化率

- さまざまな前処理を用いて Smoothing する. 収束までに要した時間
  - BJ: Block Jacobi 法
  - *-FIX: ブロックサイズを 64で固定
  - *-AUTO: いい感じに自動調整してくれる決め方
- 27 点ステンシルの結果
- DBSR がいずれの前処理よりも高速



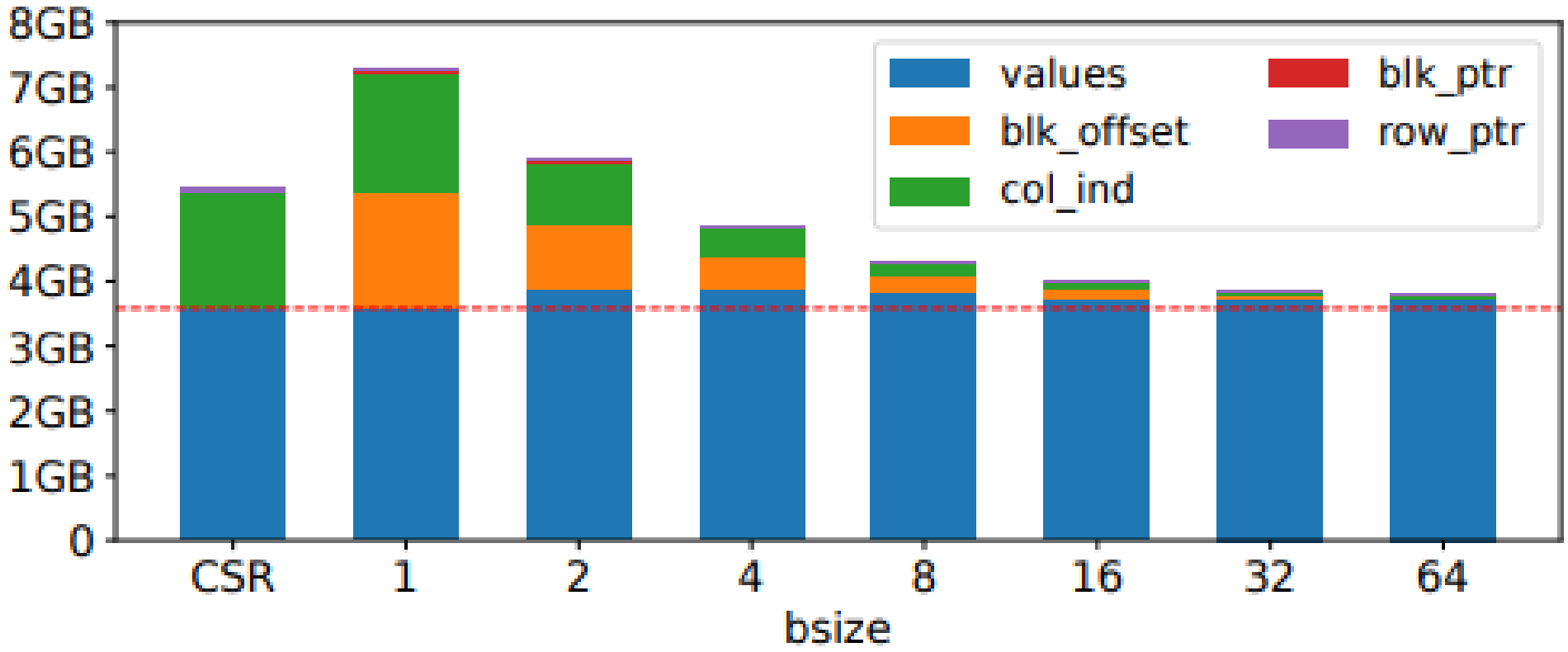
ブロックサイズの影響

- bsize を変えながら，Smoothing に必要な時間をプロット
- bsize=16 あたりで安定する
  - 並列性は上がるが, bsize を上げるほど収束性が低下するため



DBSR のメモリ使用量

- bsize を上げると, インデクス管理に使うメモリ使用量の割合が減る
  - ゼロ埋めのコストを上回る削減
  - 単精度では恩恵がより大きくなる



① SpTRSV とリオーダリング
② 格納形式と計算方法
③ DBSR 形式上での ILU(0)
④ 評価
⑤ まとめ

総括

目的

- SpTRSV の高速化
- グリッドの問題に対する最適化

手法

- VBMC アルゴリズムおよび DBSR 形式を提案
- ベクトル命令の活用

結果

- MG法の実行時間が 1.19 倍から 1.24 倍高速化（他の最新の実装比）
- 良好な弱スケーリング性
- ILU(0) を平均 1.17 倍から 1.31 倍高速化（倍精度）

所感

疑問

- VBMC ってただの Geometric MC なのでは…
  - Geometric: あらかじめ構造が分かっているとき (↔ Algebraic)
  - 実験中の MC や BMC は Algebraic に実行? ちょっと不公平

良さそうなところ

- 格納形式はよく考えられている
  - ILU(0) の vload の方法がテクい
- SpMV も速くなりそう